

Demostración · El que abre el edificio

El lab de microservicios con Docker · Guía del instructor · Curso de Spring Boot · SII
2026

15 minutos proyectando · Docker Compose · el mismo sistema del Lab de
Microservicios

Resumen

Que el sistema del Lab de Microservicios **no cambia** al meterlo en contenedores —el código es el mismo, byte a byte— y que aun así se trabaja de otra manera: **cuatro terminales y un orden que hay que recordar** pasan a ser `docker compose up`, y un servicio que se muere **vuelve solo en once segundos** sin que el usuario vea un error.

Índice

1. Antes de empezar	2
1.1. Esto no es un laboratorio	2
1.2. Qué hay que dejar preparado	2
1.3. La puesta a punto	3
2. El caso	3
2.1. El que abre el edificio, que es la metáfora de esta demostración	3
3. Los seis momentos	4
3.1. 1 · Que funciona igual	4
3.2. 2 · El arranque, que es el motivo de todo esto	4
3.3. 3 · Las bases dejan de ir dentro del programa	6
3.4. 4 · Se hablan por nombre	7
3.5. 5 · Matar un contenedor, que es el momento	8
3.6. 6 · Seguir una petición por cuatro contenedores	9
3.7. 7 · El cierre	10
4. Lo que aprendiste	11
5. Para profundizar	11
6. Antes de cerrar	12

1 Antes de empezar

1.1 Esto no es un laboratorio

El alumno no corre nada de esto. Las máquinas del SII no tienen Docker y no pueden instalarlo — ése fue el motivo de retirar el Lab de Microservicios antiguo, que sí lo pedía. Aquí se proyecta, se mira y se vuelve al laboratorio de verdad.

Por eso esta guía **no tiene pasos para pegar**: no hay nada que teclear. Lo que tiene es **qué señalar y en qué orden**.

1.2 Qué hay que dejar preparado

Requisito	Cómo lo compruebas	Qué tiene que salir
Docker Desktop corriendo	<code>docker info</code>	No dice «Cannot connect to the Docker daemon»
Las dos imágenes base, ya bajadas	<code>docker image ls grep -E 'temurin postgres'</code>	Están eclipse-temurin:25-jre-alpine y postgres:16-alpine
El sistema construido	<code>./construir.sh && docker compose build</code>	Cuatro jar y cuatro imágenes

Si te atascas

Baja las dos imágenes base ANTES de la clase.

Son 233 MB y 388 MB, y es **lo único de esta demostración que necesita internet**. Hacerlo delante de la sala con el wifi del SII es la forma más segura de perder diez minutos y el hilo.

Después de eso, todo lo demás corre sin red: los cuatro jar se compilan con el `./mvnw` del curso en modo offline, contra `repo-maven/`.

Y una cosa que muerde si se proyecta desde otra máquina: las imágenes se construyen para la **arquitectura de la máquina donde se ejecuta docker compose build**. Una construida en un Mac con Apple Silicon es `arm64`; en un portátil Intel, `amd64`. Si preparas la demostración en una máquina y la proyectas desde otra, **reconstruye allí**. Es lo mismo que dice el Lab 13 sobre la arquitectura de destino, ahora en la práctica.

1.3 La puesta a punto

```
cd demos-instructor/microservicios-docker
./construir.sh           # los cuatro jar, sin red
docker compose up -d    # y déjalo arriba antes de que entre nadie
```

Levantarlo por primera vez delante de la sala es construir cuatro imágenes en directo. Que esté arriba. Para el momento del arranque —que sí hay que enseñar— se hace `docker compose down` y `up` otra vez: son 21 segundos y se ven bien.

2 El caso

La guía del Lab de Microservicios dejó a la DGT partida en **cuatro oficinas**: Contribuyentes con su padrón, Trámites con sus expedientes, Auditoría con su registro, y una **recepción** en la calle por la que entra el ciudadano. Cada una con **su propio archivador**, que es lo que las hace cuatro oficinas y no un mostrador con cuatro ventanillas.

Funciona. Y abrir por la mañana es un trabajo.

2.1 El que abre el edificio, que es la metáfora de esta demostración

Hasta hoy, las cuatro oficinas las abría el jefe de cada una, y había que ponerse de acuerdo.

Contribuyentes tiene que abrir antes que Trámites —si no, Trámites descuelga el teléfono y no contesta nadie—. La recepción, la última, cuando ya hay a quién dirigir. Ese orden **no está escrito en ninguna parte**: está en la cabeza de quien abre, y el día que se equivoca, las primeras diez personas se van con la respuesta a medias.

Hoy hay un conserje. Uno solo, y hace cuatro cosas:

- **Abre en orden**, y el orden lo tiene escrito. No lo recuerda: lo lee.
- **No abre la siguiente hasta que la anterior contesta de verdad.** No espera «cinco minutos por si acaso»: llama a la puerta y espera a que le abran.
- **Reparte un directorio por nombre.** Nadie tiene que saber en qué despacho está Contribuyentes hoy; se pregunta por «Contribuyentes» y ya.
- **Si una ventanilla se cierra sola, la vuelve a abrir.** Sin que nadie se lo pida y sin llamar a nadie por teléfono.

Y ahora lo que **NO** hace el conserje, que es la mitad de esta demostración:

No atiende a nadie. No sabe tramitar, no conoce el padrón y no decide qué

contestarle a un ciudadano cuando la oficina de al lado no coge el teléfono. **Eso sigue siendo del que trabaja.**

3 Los seis momentos

3.1 1 · Que funciona igual

Qué vamos a hacer

Quitar de en medio la sospecha de que esto es otro sistema.

Qué señalar

Lo mismo que en el laboratorio, con el mismo token y la misma ruta:

```
TOKEN=$(curl -s -X POST http://localhost:8220/auth/login \
-H 'Content-Type: application/json' \
-d '{"usuario":"carolina","clave":"dgt2026"}' | sed
↪ 's/.*"token":\\"([^\"]*)\\".*\/1/' )

curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8220/tramites/1
```

Lo que vas a ver

```
{"id":1,"tipo":"DECLARACION_F29","estado":"EN_PROCESO",
"rutContribuyente":"11111111-1","nombreContribuyente":"Carolina Fuentes Aravena",
"estadoDelNombre":"OK","creadoEn":"2026-09-01T03:56:08.360599Z"}
```

La frase: «Es la misma respuesta del laboratorio. Ese nombre no está en la base de trámites: vino por HTTP desde otro contenedor, exactamente como venía de otra terminal. El código es el mismo —hay un verificador que lo comprueba archivo a archivo—. Lo único que cambió es cómo se arranca.»

Vas bien si...

estadoDelNombre dice OK y el nombre está lleno.

3.2 2 · El arranque, que es el motivo de todo esto

Qué vamos a hacer

Tirar el sistema entero y levantarlo delante de la sala, mirando el log.

```
docker compose down
docker compose up
```

Para entenderlo mejor

Es el conserje abriendo el edificio. Va oficina por oficina, en orden, y **llama a la puerta antes de abrir la siguiente.**

El problema

El README del laboratorio dice el orden a mano —contribuyentes, trámites, auditoría, gateway— y advierte de lo que pasa si te equivocas: las primeras llamadas fallan mientras el vecino todavía arranca, el circuit breaker las cuenta como fallos **porque lo son**, y abre. Un sistema entero sano, recién levantado, respondiendo degradado durante **18 segundos**.

Eso le pasa a todo el mundo en producción, **en cada despliegue**.

La alternativa, y por qué no

- **Acordarse del orden.** Es lo del laboratorio, y funciona hasta que alguien tiene prisa.
- **Un sleep 30 entre servicio y servicio.** Funciona en la máquina de quien lo escribió y falla en la del vecino con menos RAM. Y cuando funciona, sobra tiempo.
- **Esperar al HEALTHCHECK de verdad**, que es lo de aquí: espera lo que haga falta y ni un segundo más.

La técnica

```
build:
  context: .
  dockerfile: Dockerfile
  args:
    MODULO: tramites
mem_limit: 448m
environment:
  DB_USUARIO: svc_tramites
  DB_CLAVE: tramites-dev
networks: [red-dgt, datos-tramites]
ports:
  - "8222:8080"
depends_on:
  db-tramites:
    condition: service_healthy
  contribuyentes:
    condition: service_healthy
  auditoria:
    condition: service_healthy
```

depends_on con condition: service_healthy es toda la técnica. Y los tres healthcheck a los que consulta preguntan por /salud, **que el laboratorio ya traía**: no hizo falta añadir nada al código para esto.

Lo que vas a ver

✓ Container microservicios-docker-db-contribuyentes-1	Healthy
✓ Container microservicios-docker-db-tramites-1	Healthy
✓ Container microservicios-docker-db-auditoria-1	Healthy
✓ Container microservicios-docker-contribuyentes-1	Healthy
✓ Container microservicios-docker-auditoria-1	Healthy
✓ Container microservicios-docker-tramites-1	Healthy
✓ Container microservicios-docker-gateway-1	Healthy

21 segundos, medido, desde up hasta que el gateway contesta.

La frase: «El orden está escrito una vez, aquí, y lo cumple la máquina. En el laboratorio está escrito en el README y lo cumple usted, cada vez, sin equivocarse.»

Si te atascas

Y la trampa que hay que desactivar antes de que alguien la muerda.

Esto **no** significa que el sistema *necesite* ese orden. Significa que arrancar ordenado evita el circuito abierto de los primeros segundos. **La dependencia es de arranque, no de vida** — y el momento 5 lo demuestra: con el sistema arriba, se mata contribuyentes y los demás siguen respondiendo.

3.3 3 · Las bases dejan de ir dentro del programa

Qué vamos a hacer

Enseñar los siete contenedores y abrir dos clases Java al lado.

```
docker compose ps
```

El problema

En el laboratorio, cada servicio levanta **su propia base dentro de su propio proceso**, con Zonky. Por eso ContribuyentesApplication tenía que montar su infraestructura antes de arrancar Spring, y por eso hacían falta dos guardas —de puerto y de candado—: con cuatro terminales, arrancar el mismo servicio dos veces por equivocación es lo más fácil del mundo.

La técnica

Así arranca en el laboratorio:

```
public static void main(String[] args) throws IOException {
    new MotorDePostgres(PUERTO_BASE).levantar();
    SpringApplication.run(ContribuyentesApplication.class, args);
}
```

Y así aquí:

```
public static void main(String[] args) {
    SpringApplication.run(ContribuyentesApplication.class, args);
}
```

La frase: «Un main que ya no tiene que montar su propia infraestructura. Las dos guardas y el motor embebido no se arreglaron: se volvieron innecesarios, porque cada contenedor tiene su red y su sistema de archivos.»

Y la frontera de datos, que aquí es más fuerte

```
docker compose exec tramites getent hosts db-tramites
docker compose exec tramites getent hosts db-contribuyentes
```

```
192.168.32.2      db-tramites  db-tramites
<- la segunda no devuelve nada
```

```
networks:
  red-dgt:
  datos-contribuyentes:
  datos-tramites:
  datos-auditoria:
```

Cada servicio y su base comparten una red privada **de dos miembros**. El JOIN «rápido» para ahorrarse la llamada HTTP no es que esté prohibido: **es que no hay por dónde intentarlo**.

Vas bien si...

db-tramites devuelve una IP desde trámites; db-contribuyentes, nada.

3.4 4 · Se hablan por nombre

Qué vamos a hacer

Enseñar que localhost:8211 se convirtió en contribuyentes:8080.

```
docker compose exec tramites getent hosts contribuyentes
```

Para entenderlo mejor

Es el directorio del conserje. Nadie tiene que saber en qué despacho está Contribuyentes hoy.

La técnica

```
microservicios:
  contribuyentes:
    url: http://contribuyentes:8080
  tramites:
    url: http://tramites:8080
  auditoria:
    url: http://auditoria:8080
  jwt:
    secreto: ${LAB14_JWT_SECRET0:clave-de-laboratorio-que-debe-tener-32-bytes-o-mas}
```

En el laboratorio eso mismo dice http://localhost:8211, :8212 y :8213. **Lo único que cambió es lo que hay antes de los dos puntos.**

Lo que vas a ver

```
172.31.0.3      contribuyentes  contribuyentes
```

La frase: «Esa IP cambia en cada arranque y a nadie le importa. Esto es el service discovery que el Lab de Microservicios antiguo montaba con un Eureka —un servicio más que arrancar, configurar y operar—, hecho aquí por la plataforma y sin escribir una línea.»

Y el segundo detalle, que se pasa por alto: **los cuatro escuchan en el 8080**. Repartir puertos para que no choquen era un problema de tener todo en la misma máquina.

3.5 5 · Matar un contenedor, que es el momento

Es el momento que justifica la demostración entera. Y tiene una sorpresa, así que se hace en dos actos y **en este orden**.

Acto 1 · Lo que NO pasa

```
docker compose kill contribuyentes
docker compose ps -a
```

```
contribuyentes      Exited (137) 52 seconds ago
```

Sigue muerto. El restart: unless-stopped está puesto y no ha hecho nada.

La frase: «Una política de reinicio responde a que el proceso se muera, no a que un operador mate el contenedor. Docker distingue las dos cosas a propósito: si lo mató usted, es porque quería que se quedara muerto. Un orquestador que se lo resucitara sería un orquestador con el que no se puede trabajar.»

Y mientras tanto, el sistema responde:

```
{... "nombreContribuyente":null, "estadoDelNombre":"NO_DISPONIBLE" ...}
```

HTTP 200. Es el circuit breaker del laboratorio, intacto. **El orquestador no lo sustituye.**

Acto 2 · Lo que sí pasa

Ahora se mata **el proceso, desde dentro** — que es lo que ocurre de verdad cuando algo reventita, se queda sin memoria o el kernel lo mata:

```
docker compose up -d contribuyentes
docker compose exec contribuyentes sh -c 'kill -TERM 1'
```

Lo que vas a ver

t+00s	estadoDelNombre=NO_DISPONIBLE	contribuyentes: Up 47 seconds (healthy)
t+03s	estadoDelNombre=NO_DISPONIBLE	contribuyentes: Up 2 seconds (health: starting)
t+07s	estadoDelNombre=NO_DISPONIBLE	contribuyentes: Up 6 seconds (healthy)
t+11s	estadoDelNombre=OK	contribuyentes: Up 10 seconds (healthy)

El servicio se murió y volvió solo en once segundos, y el usuario nunca vio un error.

Tres cosas que señalar, y las tres importan:

1 · A los tres segundos ya hay un contenedor nuevo. Fíjate en el contador de Up: se reinició. Eso es lo que cuatro terminales no hacen — en el laboratorio, un servicio que muere se queda muerto hasta que alguien lo nota y vuelve a teclear `./mvnw spring-boot:run`.

2 · Entre t+07s y t+11s el contenedor ya estaba sano y la respuesta seguía degradada. Esos cuatro segundos son el **circuit breaker**, que todavía no había vuelto a probar. El orquestador levanta el proceso; **decidir cuándo volver a confiar en él sigue siendo del programa.**

3 · Todo el rato, HTTP 200. Las dos protecciones son distintas y complementarias: el circuit breaker tapa el hueco mientras dura, el reinicio hace que dure poco. **Ninguna sustituye a la otra**, y ésta es la frase con la que hay que salir de este momento.

Si te atascas

Si kill -TERM 1 no mata nada.

En algunas versiones, el proceso 1 de un contenedor ignora las señales que no tiene manejadas. La JVM sí maneja SIGTERM —es lo que dispara los *shutdown hooks* de Spring—, así que aquí funciona. Si en tu máquina no, sirve igual `docker compose restart contribuyentes` para enseñar la vuelta, aunque entonces el reinicio lo pediste tú y se pierde media lección.

3.6 6 · Seguir una petición por cuatro contenedores

Éste es el bloque que paga el Lab 11. Allí se puso un `traceId` por petición y se dijo que servía para cruzar de un servicio a otro. Aquí se ve cruzando.

Qué vamos a hacer

Mandar **una** petición con un id inventado y encontrarlo en los cuatro servicios.

La técnica

Tiene que ser un POST, y conviene saber por qué antes de teclearlo: auditoría sólo se entera cuando se **crea** un trámite. Un GET `/tramites/1` cruza tres servicios; el POST cruza los cuatro.

```
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H "X-Trace-Id: DEMO-1" \
  -H 'Content-Type: application/json' \
  -d '{"rutContribuyente":"11111111-1","tipo":"DECLARACION_F29"}' \
  http://localhost:8220/tramites > /dev/null
```

```
docker compose logs --no-color | grep DEMO-1
```

Lo que vas a ver

```
gateway-1      | 22:33:16.305 INFO [DEMO-1] GATEWAY - [GATEWAY] POST /tramites ->
↳ tramites
tramites-1     | 22:33:16.339 INFO [DEMO-1] TRAMITES - [TRAMITES] trámite 3 creado
↳ para 11111111-1
tramites-1     | 22:33:16.342 INFO [DEMO-1] TRAMITES - [TRAMITES] pido la ficha de
↳ 11111111-1 a contribuyentes
contribuyentes-1 | 22:33:16.345 INFO [DEMO-1] CONTRIBUYENTES - [CONTRIBUYENTES] me
↳ piden la ficha de 11111111-1
auditoria-1    | 22:33:16.371 INFO [DEMO-1] AUDITORIA - [AUDITORIA] llega el evento
↳ TRAMITE_CREADO del trámite 3 - procesando...
auditoria-1    | 22:33:17.959 INFO [DEMO-1] AUDITORIA - [AUDITORIA] REGISTRADO id=1
↳ del trámite 3
tramites-1     | 22:33:17.965 INFO [DEMO-1] TRAMITES - [TRAMITES] auditoría acusó
↳ recibo del trámite 3
```

Cuatro contenedores, siete líneas, una sola historia — y en orden de reloj, aunque cada línea la escribió un proceso distinto.

Qué señalar, en este orden

1 · El id lo puso quien llamó. Con `-H "X-Trace-Id: DEMO-1"`. El gateway lo **respetó** en vez de inventarse uno, y eso es el `if` de tres líneas del `FiltroDeCorrelacion` del Lab 11:

```
String id = petition.getHeader(CABECERA);
if (id == null || id.isBlank()) {
    id = UUID.randomUUID().toString().substring(0, 8);
}
```

Aquel día parecía una cortesía. Aquí es lo que hace posible todo lo demás.

2 · Cruzó de proceso a proceso porque cada cliente HTTP lo reenvía: `Enrutador` en el gateway, `ClienteContribuyentes` y `ClienteAuditoria` en trámites. Y el filtro del otro extremo lo recoge. **Nadie lo pasa por parámetro en ningún método de negocio** — el `TramiteController` no menciona el `traceId` por ninguna parte.

3 · Y en `ClienteAuditoria` hay un detalle que está a la vista en la salida: fijate en el **segundo y medio** entre llega el evento (...16.371) y `REGISTRADO` (...17.959), y en que `tramites` no dice auditoría acusó recibo hasta después. Esa llamada es asíncrona, así que el `id` se copia del `MDC` **antes** de saltar al otro hilo.

```
String traceId = MDC.get(FiltroDeCorrelacion.CLAVE);
// ... y dentro del otro hilo:
MDC.put(FiltroDeCorrelacion.CLAVE, traceId);
```

El `MDC` va atado **al hilo**. En el hilo nuevo está vacío. Es exactamente la trampa que el Lab 12 dejó anunciada con `@Async`, y aquí se ve resuelta.

Y el contraste, que es por qué este bloque existe

En el laboratorio esto son **cuatro terminales** que hay que mirar a la vez, buscando el mismo `id` a ojo en cuatro consolas que scrollean.

Aquí es **un grep**.

Eso es la mitad de la respuesta a la pregunta que quedó abierta en el Lab 11 — «*contarlo, ¿a quién?*». `Compose` junta las salidas de los siete contenedores en un solo flujo; en producción ese trabajo lo hace un recolector (`Loki`, `Elasticsearch`, el del proveedor), y entonces el `grep` es una barra de búsqueda sobre semanas de tráfico y cientos de máquinas.

Si te atascas

Si `grep DEMO-1` no devuelve nada.

Comprueba que `$TOKEN` sigue vivo —el del bloque 1 caduca— y que la cabecera va escrita exactamente `X-Trace-Id`. Si sólo aparece la línea del gateway, el reenvío se rompió en algún cliente: mira que `ClienteContribuyentes` y `ClienteAuditoria` sigan poniendo la cabecera.

3.7 7 · El cierre

```
docker compose down -v
```

El -v se lleva también las tres bases. Sin él, quedan tres volúmenes ocupando disco.

La frase con la que hay que cerrar, y es lo contrario de una venta:

«El orquestador no arregló nada del sistema. Lo que hizo fue quitar de en medio el trabajo de operarlo: el orden de arranque, la espera a que el vecino esté listo, las direcciones que cambian, y levantar lo que se cayó. El fallo en cascada, el circuit breaker y la decisión de qué devolver cuando no hay respuesta siguen siendo suyos.»

4 Lo que aprendiste

1 · Un orquestador es un conserje, no un programador.

Abre en orden, espera a que estén listos, reparte el directorio y vuelve a abrir lo que se cerró. No decide qué contestarle a un ciudadano cuando la oficina de al lado no coge el teléfono.

2 · El código no cambió, y está comprobado.

`tools/verificar-demo-docker.py` compara los dos árboles: **32 archivos idénticos byte a byte**, 11 con una diferencia declarada y 9 retirados a propósito. Las diferencias son exactamente las piezas que el orquestador reemplaza — el motor embebido, las dos guardas, las direcciones.

3 · Un id por petición convierte cuatro consolas en un grep.

El `traceId` del Lab 11 cruzó los cuatro servicios sin que ningún método de negocio lo mencione: lo pone un filtro, lo reenvían los clientes, lo recoge el filtro del otro extremo. En cuatro terminales eso se busca a ojo; en un flujo único, se busca con una palabra.

4 · Las dos protecciones son distintas.

El circuit breaker tapa el hueco **mientras** el vecino está caído. El reinicio automático hace que ese hueco **dure once segundos** en vez de hasta que alguien se dé cuenta. Quitar cualquiera de las dos deja un sistema peor.

5 · Y esto también se paga.

Docker Desktop, un compose que mantener, unas imágenes que reconstruir y actualizar, y una pieza más que puede fallar. Con un servicio no compensa. Con siete contenedores, compensaba antes de terminar de contarlos. **La pregunta de siempre: qué estás pagando y a cambio de qué.**

5 Para profundizar

- **Levanta el laboratorio y la demostración a la vez** y ponlos al lado. Los puertos no chocan: el lab va en 820x/821x y esto en 822x.
- **Quita un depends_on de trámites**, levanta, y mira el log del circuit breaker abriéndose en el arranque. Es el defecto que el orden previene.
- **Sube mem_limit de un servicio a 128m** y mira qué hace la JVM. Es la razón por la que el techo está puesto.
- **Añade un cuarto servicio** al compose y cuenta cuántas líneas cuesta. Compáralo con abrir una quinta terminal.
- **Manda dos peticiones con el mismo X-Trace-Id** y mira el grep. ¿Se distinguen? Es el argumento de por qué el id lo genera el que entra, y no el que atiende.

6 Antes de cerrar

```
docker compose down -v  
docker image prune -f      # solo si hace falta el disco
```

Lo que te llevas:

Un orquestador no hace mejor el sistema: hace más barato operarlo. El orden de arranque, las direcciones y los reinicios dejan de estar en la cabeza de alguien y pasan a estar escritos. Lo que se rompe cuando el vecino se cae sigue siendo tuyo. Y una petición que cruza cuatro procesos se sigue con un grep, porque alguien puso un id en el Lab 11.